



# Findings-to-Fix

Fix confirmed Checkmarx findings from VS Code with GitHub Copilot



You ask Copilot to fix your findings. Checkmarx generates the fix. Copilot shows you the change in your editor, and you click Keep or Undo.

## What you need

- **A Checkmarx One API key** on your machine, in your shell profile or in the `cx` CLI configuration.
- **Python 3 or Node** on your machine. Either one works.
- **VS Code 1.104 or newer** with the GitHub Copilot Chat extension.

## Contents

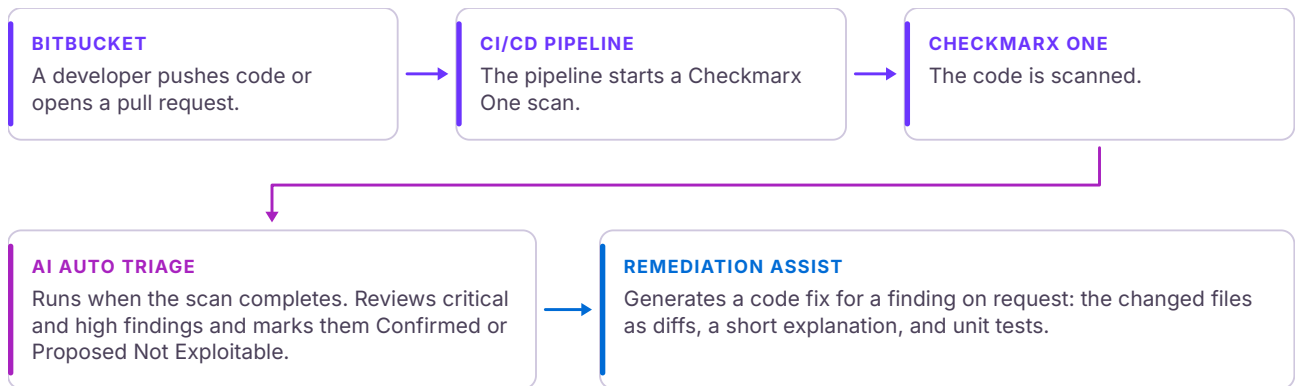
|                                       |   |
|---------------------------------------|---|
| How it fits together                  | 2 |
| Getting started                       | 3 |
| Using it                              | 4 |
| What happens, step by step            | 5 |
| Where things live and what can happen | 6 |
| Rolling out to a team                 | 7 |

Only findings that Checkmarx AI Auto Triage has marked **Confirmed**, at critical or high severity, are fixed. Nothing is committed or pushed. You review, then commit as usual.

# How it fits together

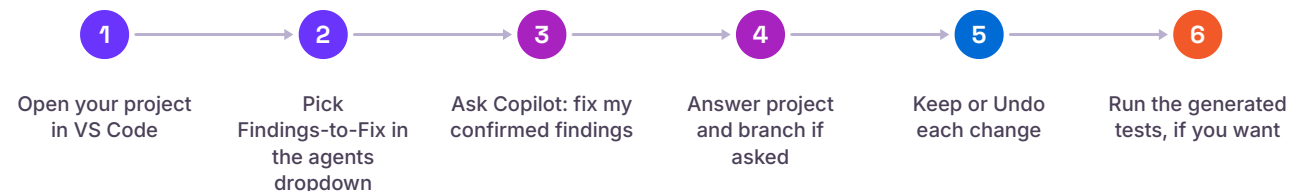
Three views of the same flow: what the platform does after a scan, what a developer does in the editor, and the path a single fix takes.

## 1 After a scan



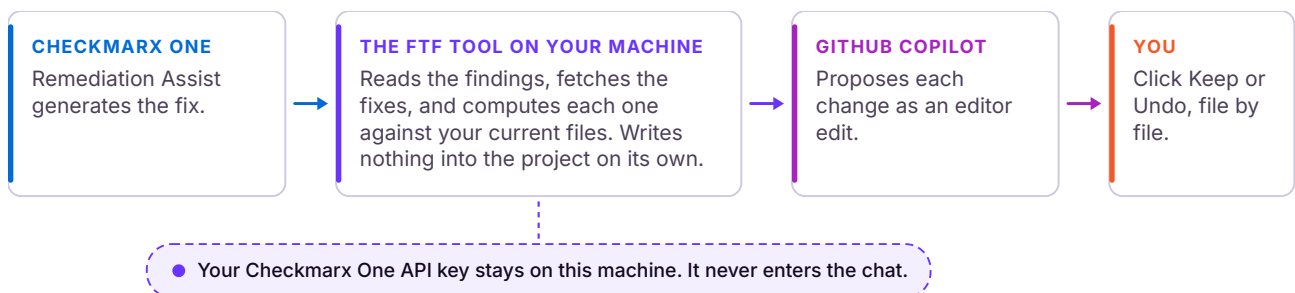
All of this happens on the Checkmarx One platform. No developer action is needed for any of it.

## 2 What the developer does



Six steps in Copilot Chat. Step 2 is optional: asking the default agent to fix your confirmed Checkmarx findings also works.

## 3 How a fix travels



Every change to code arrives as an editor edit that you accept. The plugin never commits or pushes.

# Getting started

Three things once. After that it is just asking Copilot.

## 1 A Checkmarx One API key on your machine

Ask your Checkmarx admin for an API key. Then either put it in your shell profile:

```
export CX_APIKEY=<your key>
```

or, if you already use the Checkmarx `cx` command line tool:

```
cx configure set --prop-name cx_apikey --prop-value <your key>
```

Findings-to-Fix reads it from either place. It never shows the key to Copilot.

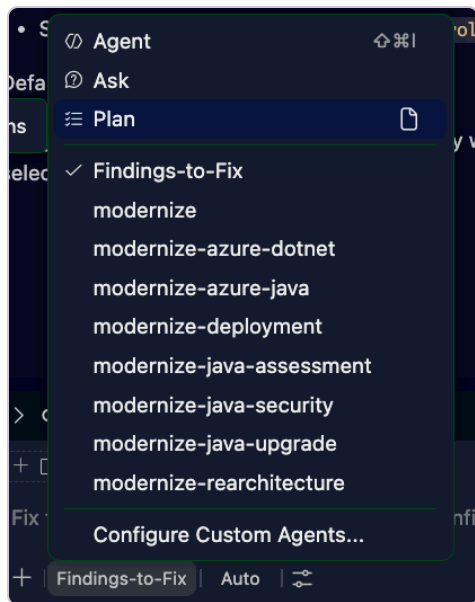
## 2 Python or Node on your machine

Either one works. Most Macs and Linux machines already have Python 3. On Windows, if you have `python3` or `node` in a terminal, you are set.

## 3 Install the plugin in VS Code

1. Open VS Code (1.104 or newer) with the GitHub Copilot Chat extension.
2. Press `Cmd+Shift+P` (Mac) or `Ctrl+Shift+P` (Windows), type **Chat: Install Plugin From Source**, press Enter.
3. Paste the plugin repository's Git URL and press Enter.
4. Reload the window when VS Code asks.

You will see a short "Findings-to-Fix is installed" note when a chat session starts.



Pick Findings-to-Fix in the agents dropdown

# Using it

Open a project, ask, then review each change.

## 1 Open the project you want to fix in VS Code

## 2 Pick Findings-to-Fix in the agents dropdown

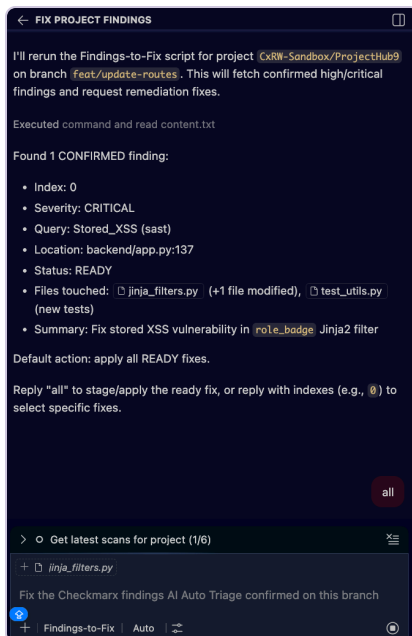
The dropdown sits at the bottom of the Copilot Chat box. If you forget, asking the default agent to "fix my confirmed Checkmarx findings" also works.

## 3 Type: **fix my confirmed findings**

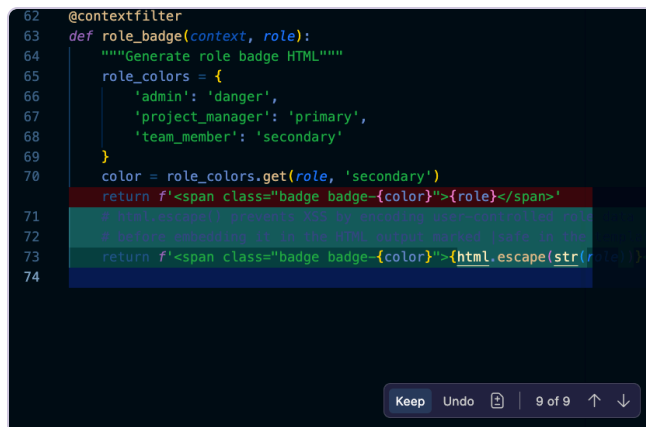
### Copilot will

- work out which Checkmarx project and branch this is. If it cannot tell from your git setup, it lists the likely projects and asks you to pick one, then the branches that have scans. It never guesses.
- fetch the fixes from Checkmarx: a couple of minutes the first time, seconds after that.
- show you a table of the confirmed findings and ask which to apply.
- open each changed file in your editor with the change shown as a diff. Click **Keep** or **Undo** on each one.
- explain what each fix does and why.
- offer to run the tests Checkmarx generated for the fix.

Nothing is committed or pushed. You review, then commit as usual.



Copilot lists the confirmed findings and asks which to apply



Each changed file opens with the change shown; click Keep or Undo

# What happens, step by step

The full sequence, from the request in chat to the commit.

- 1 **The developer asks.** In Copilot Chat, with the Findings-to-Fix agent selected, the developer types "fix my confirmed findings".
- 2 **Copilot runs the tool.** The tool reads the project's git remote and current branch and looks the project up in Checkmarx One.
- 3 **The tool asks when it is unsure.** If the name matches no project, or the branch has no completed scan, Copilot shows a numbered list and the developer picks.
- 4 **The tool fetches the fixes.** It lists the Confirmed critical and high findings on the latest completed scan and sends them to Remediation Assist in one request.
- 5 **Copilot shows the findings** as a table and asks which to apply.
- 6 **The tool computes each fix against the current files** without writing anything, producing the complete new content for each changed file.
- 7 **Copilot proposes each fix as an editor edit.** The changed file opens with the change shown as a diff and Keep and Undo controls.
- 8 **Files that changed since the scan are placed by hand,** so the developer's own edits stay in place.
- 9 **Copilot explains each fix** (what, why, how) and names the tests Remediation Assist generated.
- 10 **Copilot offers to run those tests** and does so only if the developer says yes. The developer then commits as usual. The plugin never commits or pushes.

## When a file changed since the scan

Copilot reads the intended change and the current file, makes the same change where the code now lives, keeps the developer's other edits, and says plainly that this fix was placed by hand and deserves a closer look. Copilot never overwrites a whole file on its own. Fresh fixes take two to three minutes. Fixes that already exist come back in seconds.

## What you will see

```
You:      fix my confirmed findings
Copilot:  Project CxRW-Sandbox/ProjectHub9, branch feat/update-routes,
          scan from 2026-08-13. 1 confirmed critical/high finding.
          | # | Severity | Finding      | Location          | Files touched      |
          | 0 | CRITICAL | Stored_XSS   | backend/app.py:137 | jinja_filters.py, test_utils.py |
          Apply all?
You:      yes
Copilot:  [two files open in the editor with the changes shown; Keep / Undo on each]
          backend/utills/jinja_filters.py  +4/-1  escape user text before rendering
          backend/tests/test_utils.py      +147/-0  tests for the fix
          What: ... Why: ... How: ...
          Checkmarx generated tests in backend/tests/test_utils.py. Run them?
```

# Where things live and what can happen

Which credential is used where, and the boundaries the plugin works inside.

## Where the credential goes and does not go

| Credential                           | Lives                                                          | Used by                                                                         | Never reaches                  |
|--------------------------------------|----------------------------------------------------------------|---------------------------------------------------------------------------------|--------------------------------|
| Checkmarx One API key                | shell profile or <code>cx</code> CLI config on the workstation | the <code>ftf</code> tool, which exchanges it for a short-lived token in memory | Copilot's conversation, GitHub |
| GitHub Copilot session               | VS Code's GitHub sign-in                                       | Copilot                                                                         | Checkmarx                      |
| CI/CD credential that triggers scans | the CI/CD system                                               | the pipeline                                                                    | the developer's machine        |

### What can and cannot happen

- The plugin reads findings and fetches fixes. It cannot change a finding's state, trigger a scan, or touch Checkmarx configuration.
- Every change to code arrives as an editor edit the developer accepts. The tool writes nothing into the project on its own during the normal flow.
- The plugin never commits or pushes.
- Only Confirmed critical and high findings are considered. Package (SCA) fixes are supported and off by default.
- Nothing runs on a server. Nothing is installed besides the plugin folder.

### Good to know

- It works whether your project folder is a git clone or a plain folder.
- Ask Copilot to "include package fixes" when you want SCA fixes as well.
- If something is missing (no API key, expired key, feature not enabled on your tenant), Copilot tells you exactly what and how to fix it.
- AI Auto Triage sets the Confirmed and Proposed Not Exploitable states on the platform. Nothing in the plugin changes those verdicts.

# Rolling out to a team

Host the plugin repository somewhere developers can reach, then pick one of two install routes.

## ROUTE 1

### Each developer installs it once

In VS Code, run **Chat: Install Plugin From Source** and paste the repository URL. The plugin repository can sit in Bitbucket, GitHub, or an internal Git server.

## ROUTE 2

### Copilot Business or Enterprise

An administrator adds the repository to `managed-settings.json` under `extraKnownMarketplaces` and lists the plugin under `enabledPlugins`. It then installs for everyone automatically.

## Per developer, one time

- A Checkmarx One API key.
- Python 3 or Node on the machine.
- Copilot's terminal tool allowed by the organization's Copilot policy. That is how the plugin runs.

## What is in the plugin

| Path                                         | What it is                                                             |
|----------------------------------------------|------------------------------------------------------------------------|
| plugin.json                                  | Tells VS Code this is a plugin                                         |
| agents/findings-to-fix.agent.md              | The Findings-to-Fix entry in the agents dropdown and its instructions  |
| skills/fix-confirmed-findings/SKILL.md       | The same instructions for the default agent                            |
| skills/fix-confirmed-findings/ftf.py, ftf.js | The tool that talks to Checkmarx (Python and Node versions, identical) |
| hooks/                                       | The one-line "installed" note at session start                         |
| docs/                                        | Architecture and how-it-works documents                                |

Nothing runs on a server. Nothing is installed besides the plugin folder.

## Support

Questions and issues: open an issue in the plugin repository.